

# Applied Software Project Report

---

## E-Commerce Microservices Platform (Java 17, Spring Boot 3 & Cloud)

---

### Abstract

---

**Title:** A Scalable, Secure E-Commerce Platform Built on Spring Boot Microservices.

Modern online retail is dominated by traffic spikes, flash sales, and the need to evolve individual capabilities (catalog, checkout, payments) independently. Legacy monolithic storefronts struggle here: a single overloaded feature can bring the whole site down, and every small change forces a full redeployment. This project addresses that problem by implementing a distributed e-commerce backend as seven independent microservices.

**Purpose.** The goal is to demonstrate how a retail platform can be decomposed into autonomous, independently deployable services that scale, fail, and evolve in isolation — the same architectural pattern used by large-scale retailers to survive peak shopping events.

**Methods.** The system is built with Java 17 and Spring Boot 3.2.5. Netflix Eureka provides service discovery, Spring Cloud Gateway is the single external entry point, and OpenFeign handles inter-service calls over a load-balanced service registry. Security uses stateless JWT bearer tokens issued by a user service and validated at each service, plus an internal service token for privileged machine-to-machine calls. Each service owns its own PostgreSQL database (database-per-service), with schema managed by Flyway. Order placement is coordinated as a saga with compensating transactions to keep inventory and payments consistent.

**Results.** The platform routes external traffic through a unified gateway to five business services (user, product, cart, order, payment), enforces per-request authentication, reserves and restores stock transactionally, and orchestrates mock payments with automatic rollback on failure. A unified Swagger UI documents every service API.

**Conclusions.** The microservices-plus-gateway pattern delivers independent scalability, fault isolation, and faster iteration. The same blueprint applies beyond retail — to banking, logistics, healthcare scheduling, and any domain that must scale components independently and modify existing processes without large-scale rewrites.

# Project Description

---

## Overview

The E-Commerce Microservices Platform is a backend system that powers the core shopping journey: a customer registers, browses a product catalog, adds items to a cart, places an order, and pays for it. Rather than a single application, each of these concerns is a separate service that can be deployed, scaled, and maintained on its own. Business milestones along that journey (registration, order confirmation, payment outcome, cancellation, low stock) are broadcast as events over RabbitMQ and turned into customer and operational notifications by a dedicated Notification Service.

## Objectives

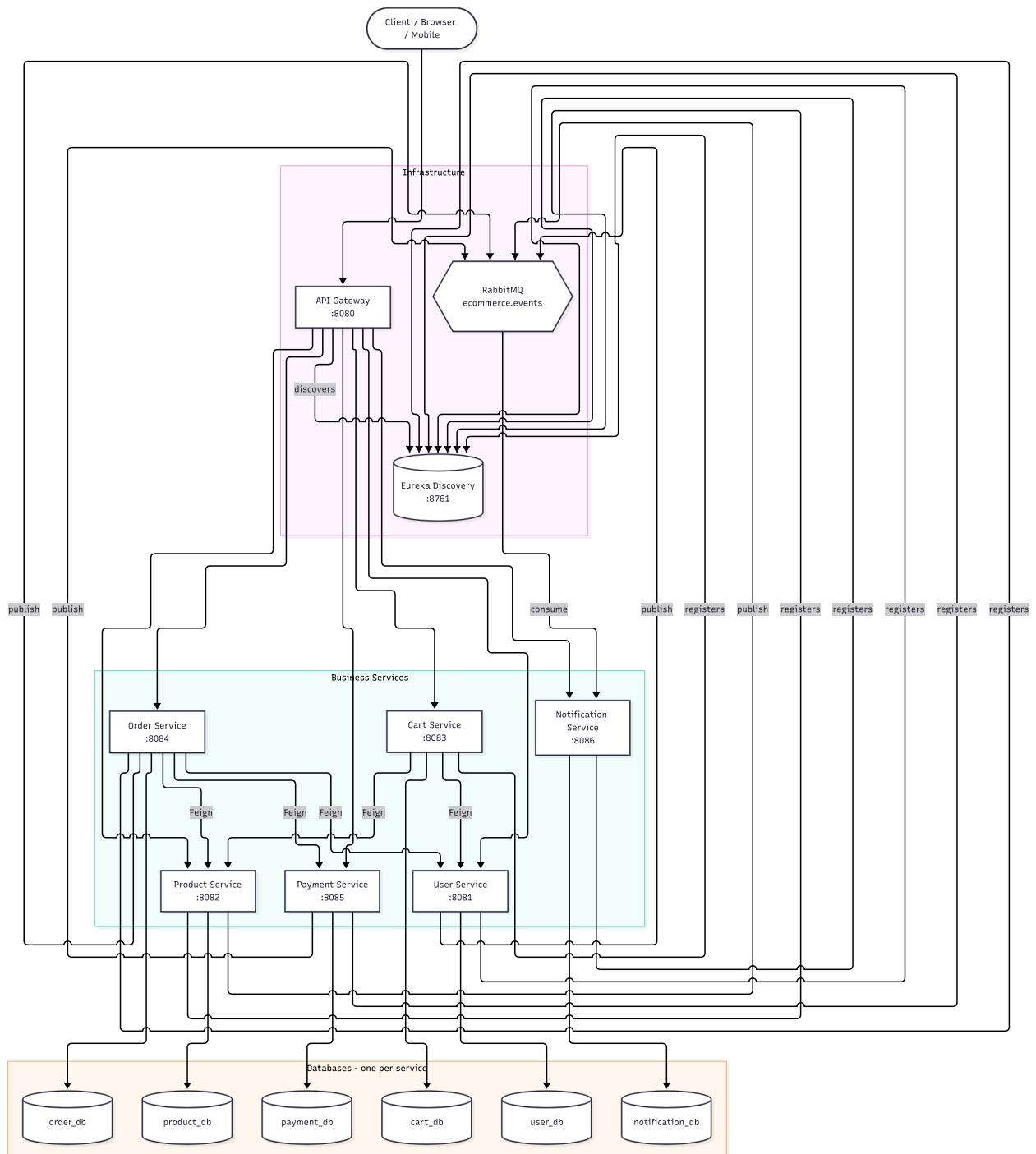
1. Decompose an e-commerce backend into cohesive, independently deployable services.
2. Provide a single, secured entry point for all client traffic (API Gateway).
3. Enable services to find and call each other dynamically via service discovery.
4. Secure every API with stateless JWT authentication and protect internal machine-to-machine endpoints with a service token.
5. Keep data consistent across services during multi-step operations (order + stock + payment) using a saga with compensation.
6. Isolate data ownership so each service manages its own database schema.
7. Decouple side effects (notifications) from the transactional path using event-driven messaging, so a slow or failing notification never blocks core commerce.

## Relevance

This design mirrors how real retailers handle Black Friday-scale load: the product catalog can scale horizontally during a sale without touching the payment service, a failure in payments does not corrupt inventory, and teams can ship changes to one service without redeploying the rest. The pattern generalizes to any industry modernizing a monolith into scalable, fault-isolated components.

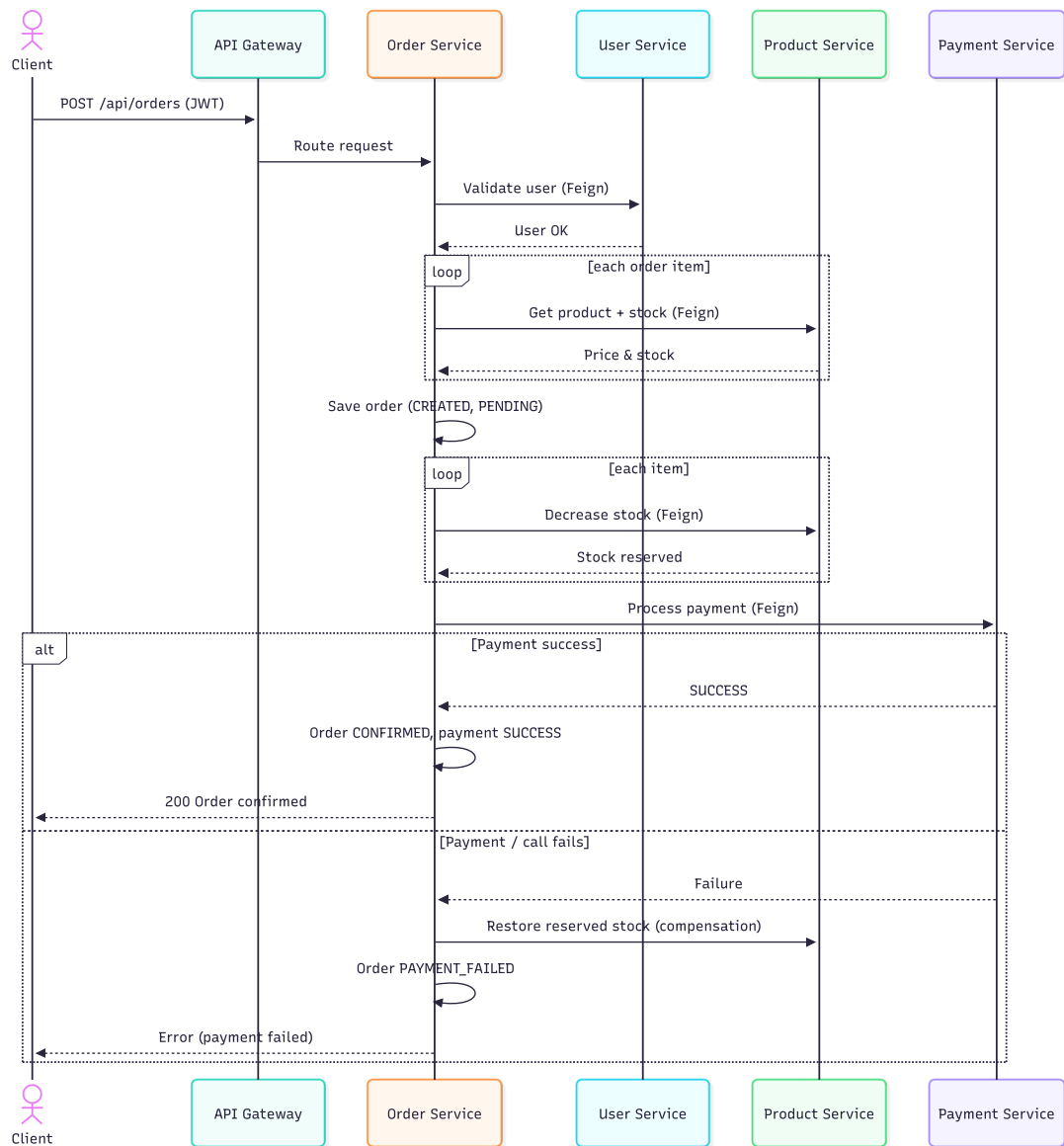
## System Architecture

Figure 1: High-level system architecture — client traffic enters through the API Gateway, services register with Eureka, services communicate synchronously via Feign, and business events flow asynchronously through RabbitMQ to the Notification Service.



## Order Placement Flow (Saga with Compensation)

Figure 2: Sequence of an order being placed. Stock is reserved, payment is attempted, and any failure triggers compensating rollback of the reserved stock.



## Technology Stack

| Layer               | Technology                                                      |
|---------------------|-----------------------------------------------------------------|
| Language / Runtime  | Java 17                                                         |
| Framework           | Spring Boot 3.2.5                                               |
| Cloud / Distributed | Spring Cloud 2023.0.1, Netflix Eureka, Spring Cloud Gateway     |
| Inter-service calls | OpenFeign (load-balanced via Eureka)                            |
| Async messaging     | RabbitMQ (Spring AMQP) — event-driven notifications             |
| Notifications       | Spring Mail (SMTP), Thymeleaf templating                        |
| Security            | Spring Security, JWT (jjwt 0.12.5), internal service token      |
| Persistence         | Spring Data JPA, PostgreSQL 16, Flyway migrations               |
| API Docs            | SpringDoc OpenAPI / Swagger UI                                  |
| Build               | Maven (independent module per service)                          |
| Deployment          | Docker Compose (services + one Postgres per service + RabbitMQ) |

# Requirement Gathering

---

## Functional Requirements

| ID    | Requirement                                                                                           |
|-------|-------------------------------------------------------------------------------------------------------|
| FR-1  | Users can register with name, email, and password; email must be unique.                              |
| FR-2  | Users can log in and receive a JWT containing their identity and role.                                |
| FR-3  | Users can view and update their own profile; a user cannot modify another user's profile.             |
| FR-4  | Admins can update a user's role (USER / ADMIN).                                                       |
| FR-5  | The catalog supports listing products, searching by keyword, and filtering by category.               |
| FR-6  | Products and categories can be created, updated, and deleted.                                         |
| FR-7  | Users can add, update, remove, and clear items in a personal cart.                                    |
| FR-8  | The cart uses authoritative product price and stock from the product service, not client input.       |
| FR-9  | Users can place an order; the system validates the user, prices items, and checks stock.              |
| FR-10 | Placing an order reserves stock and initiates payment atomically as a saga.                           |
| FR-11 | On payment failure, reserved stock is automatically restored and the order is marked PAYMENT_FAILED.  |
| FR-12 | Users can view their order history and individual orders (own orders only).                           |
| FR-13 | Users can cancel an eligible order; stock is restored and a successful payment is refunded.           |
| FR-14 | The payment service processes mock payments for supported methods and is idempotent per order.        |
| FR-15 | All external API traffic is routed through a single API Gateway.                                      |
| FR-16 | Each service exposes OpenAPI docs; the gateway aggregates them into one Swagger UI.                   |
| FR-17 | Registering a user sends a welcome notification.                                                      |
| FR-18 | Confirming an order sends an order-confirmation notification; a successful payment sends a receipt.   |
| FR-19 | A failed payment sends a payment-failure alert; a cancelled order sends a cancellation/refund notice. |
| FR-20 | When a product's stock crosses a low threshold, an operational low-stock alert is sent.               |
| FR-21 | Users can view their notification history and manage their channel preferences (email/SMS/push).      |

## Non-Functional Requirements

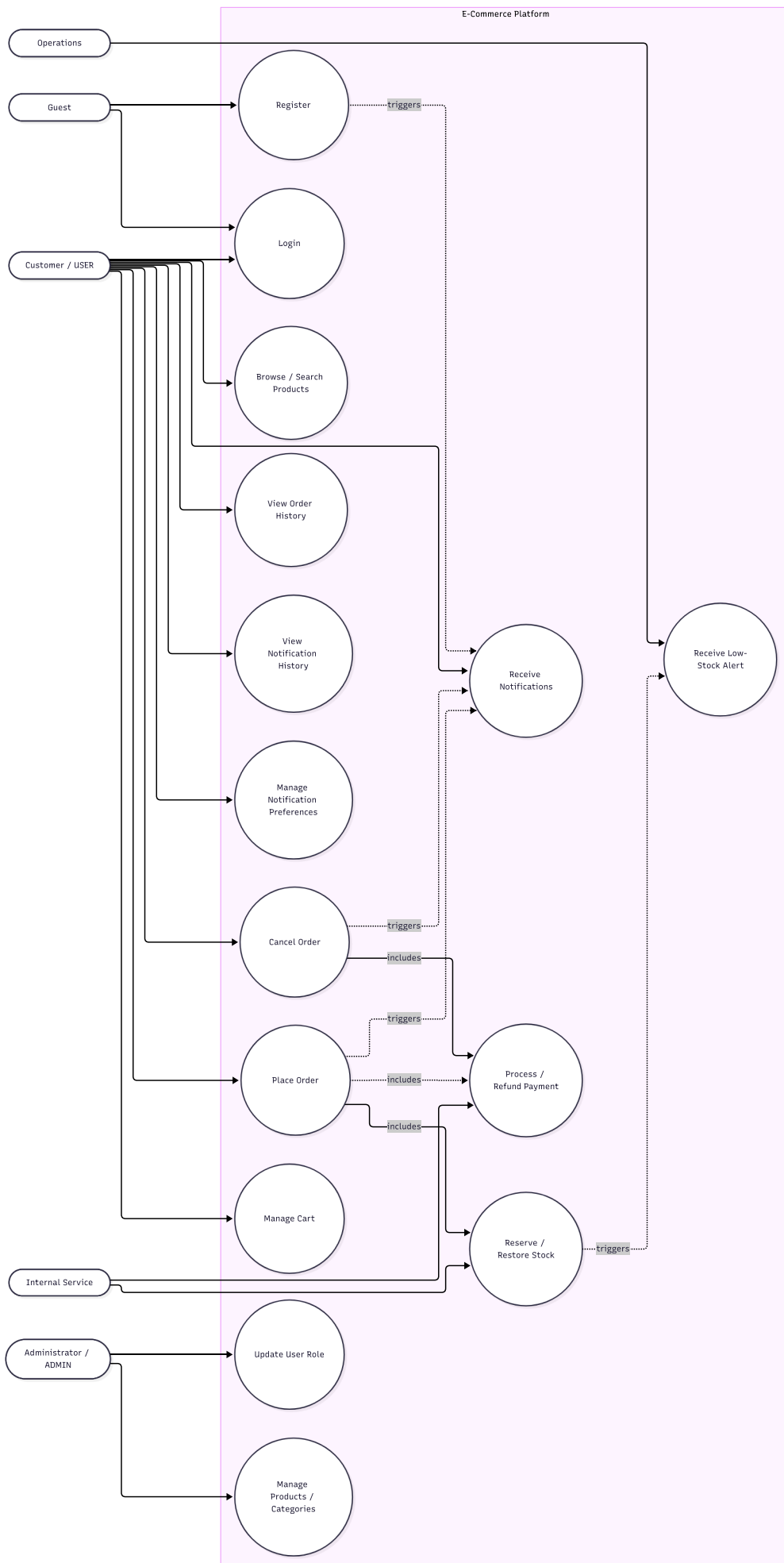
| ID     | Category                       | Requirement                                                                                                                         |
|--------|--------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| NFR-1  | Scalability                    | Each service scales independently behind the gateway and service registry.                                                          |
| NFR-2  | Availability / Fault isolation | A failure in one service (e.g. payment) must not corrupt data in others.                                                            |
| NFR-3  | Security                       | Stateless JWT bearer auth on all protected APIs; shared signing secret across services.                                             |
| NFR-4  | Security (internal)            | Privileged endpoints (stock mutation, refunds) require an internal service token.                                                   |
| NFR-5  | Data integrity                 | Stock updates use row-level locking; carts use optimistic locking ( <code>@Version</code> ).                                        |
| NFR-6  | Data isolation                 | Database-per-service; no cross-service foreign keys, only logical IDs.                                                              |
| NFR-7  | Maintainability                | Clear layered structure (controller → service → repository) per service.                                                            |
| NFR-8  | Observability                  | Spring Boot Actuator health/info endpoints on each service.                                                                         |
| NFR-9  | Portability                    | Fully containerized via Docker Compose for reproducible environments.                                                               |
| NFR-10 | Consistency                    | Multi-step order operations use saga with compensating transactions.                                                                |
| NFR-11 | Decoupling                     | Notifications are event-driven over RabbitMQ; they never block or fail the transaction that triggers them (best-effort publishing). |
| NFR-12 | Delivery reliability           | Notification delivery is idempotent per (event, channel); failed deliveries are retried and finally dead-lettered.                  |

## Users and Use Cases

### User roles

| Actor                 | Description                                                                                                         |
|-----------------------|---------------------------------------------------------------------------------------------------------------------|
| Guest                 | Unauthenticated visitor; can register and log in.                                                                   |
| Customer (USER)       | Authenticated shopper; manages cart, places/cancels orders, views own orders, and manages notification preferences. |
| Administrator (ADMIN) | Manages the product catalog and user roles.                                                                         |
| Operations            | Receives operational alerts such as low-stock notifications.                                                        |
| Internal Service      | Machine actor (e.g. Order Service) calling privileged endpoints with a service token.                               |
| Notification Service  | Event consumer that turns domain events into customer and operational notifications.                                |

Figure 3: Use Case Diagram — actors and the primary operations they perform.



## Key use case: Place Order

- **Actor:** Customer
- **Precondition:** User is authenticated (valid JWT) and has selected products.
- **Main flow:** Validate user → price and stock-check items → save pending order → reserve stock → process payment → confirm order.
- **Alternate flow:** If stock is insufficient or payment fails, restore reserved stock and mark the order **PAYMENT\_FAILED** .
- **Postcondition:** Order is **CONFIRMED** with payment **SUCCESS** , or safely rolled back.

## Key use case: Receive Notification

- **Actor:** Customer (recipient); Notification Service (system).
- **Trigger:** A domain event is published (e.g. **order.confirmed** ) after a business operation commits.
- **Precondition:** The event carries the recipient contact; the user has the relevant channel enabled in their preferences.
- **Main flow:** Notification Service consumes the event → checks idempotency → renders the message → dispatches on each enabled channel → records the result.
- **Alternate flow:** If a channel is disabled the delivery is **SKIPPED** ; if delivery fails it is retried and finally dead-lettered.
- **Postcondition:** A delivery record exists with status **SENT** , **SKIPPED** , or **FAILED** ; the transaction that triggered the event is unaffected either way.

## Feature Set

### User Service (:8081)

| Feature            | Endpoint                           | Access                       |
|--------------------|------------------------------------|------------------------------|
| Register           | <b>POST</b> /api/users/register    | Public                       |
| Login (JWT issue)  | <b>POST</b> /api/users/login       | Public                       |
| Get user by ID     | <b>GET</b> /api/users/{id}         | Authenticated                |
| Update own profile | <b>PUT</b> /api/users/{id}/profile | Owner only                   |
| Update role        | <b>PUT</b> /api/users/{id}/role    | Authenticated (admin intent) |
| Logout             | <b>POST</b> /api/users/logout      | Authenticated                |

### Product Service (:8082)

| Feature                          | Endpoint                                | Access                       |
|----------------------------------|-----------------------------------------|------------------------------|
| List / search / filter products  | GET /api/products                       | Authenticated                |
| Products by category             | GET /api/products/category/{categoryId} | Authenticated                |
| Get product                      | GET /api/products/{id}                  | Authenticated                |
| Create / update / delete product | POST/PUT/DELETE /api/products           | Authenticated (admin intent) |
| Decrease stock                   | PATCH /api/products/{id}/stock          | Internal token + JWT         |
| Restore stock                    | PATCH /api/products/{id}/stock/restore  | Internal token + JWT         |

### Cart Service (:8083)

| Feature              | Endpoint                                     | Access     |
|----------------------|----------------------------------------------|------------|
| Get / create cart    | GET /api/carts/{userId}                      | Owner only |
| Add item             | POST /api/carts/{userId}/items               | Owner only |
| Update item quantity | PUT /api/carts/{userId}/items/{productId}    | Owner only |
| Remove item          | DELETE /api/carts/{userId}/items/{productId} | Owner only |
| Clear cart           | DELETE /api/carts/{userId}                   | Owner only |

### Order Service (:8084)

| Feature             | Endpoint                         | Access        |
|---------------------|----------------------------------|---------------|
| Place order         | POST /api/orders                 | Authenticated |
| Order history (own) | GET /api/orders/user/{userId}    | Owner only    |
| Get order (own)     | GET /api/orders/{orderId}        | Owner only    |
| Cancel order        | PUT /api/orders/{orderId}/cancel | Owner only    |
| Update order status | PUT /api/orders/{orderId}/status | Authenticated |

### Payment Service (:8085)

| Feature              | Endpoint                                  | Access         |
|----------------------|-------------------------------------------|----------------|
| Process payment      | POST /api/payments/process                | Authenticated  |
| Get payment by order | GET /api/payments/order/{orderId}         | Authenticated  |
| Refund payment       | POST /api/payments/order/{orderId}/refund | Internal token |

## Notification Service (:8086)

| Feature                            | Endpoint / Trigger                            | Access        |
|------------------------------------|-----------------------------------------------|---------------|
| Notification history (own)         | GET /api/notifications/user/{userId}          | Authenticated |
| Get notification preferences       | GET /api/notifications/preferences/{userId}   | Authenticated |
| Update notification preferences    | PUT /api/notifications/preferences/{userId}   | Authenticated |
| Welcome email                      | Consumes <code>user.registered</code> event   | Event-driven  |
| Order confirmation                 | Consumes <code>order.confirmed</code> event   | Event-driven  |
| Order cancellation / refund notice | Consumes <code>order.cancelled</code> event   | Event-driven  |
| Payment receipt                    | Consumes <code>payment.succeeded</code> event | Event-driven  |
| Payment failure alert              | Consumes <code>payment.failed</code> event    | Event-driven  |
| Low-stock ops alert                | Consumes <code>product.low_stock</code> event | Event-driven  |

## Infrastructure

| Feature           | Component                     | Detail                                                           |
|-------------------|-------------------------------|------------------------------------------------------------------|
| Service discovery | Eureka (:8761)                | Dynamic registration and lookup                                  |
| API routing       | Spring Cloud Gateway (:8080)  | Single entry point, path-based routes                            |
| Async messaging   | RabbitMQ (:5672, mgmt :15672) | <code>ecommerce.events</code> topic exchange + dead-letter queue |
| Unified API docs  | SpringDoc                     | Aggregated Swagger UI at <code>/swagger-ui.html</code>           |
| Containerization  | Docker Compose                | All services + one Postgres per service + RabbitMQ               |

## Class Diagrams (Low-Level Design)

This section documents the Low-Level Design (LLD) of the platform: the classes, their fields and key methods, and the relationships between them within each service. Because every service owns a separate database, associations that cross a service boundary are **logical references** (dashed) carried as plain ID fields, while associations owned by a single JPA context are **structural** (solid).

**Note on figures.** The diagrams below are authored as Mermaid `classDiagram` definitions so they render directly in GitHub, VS Code, and Kiro. Per the project guideline, editable **draw.io** versions are maintained under `docs/diagrams/` and exported as PNGs. To embed an exported image, follow the standard figure format used throughout this report:

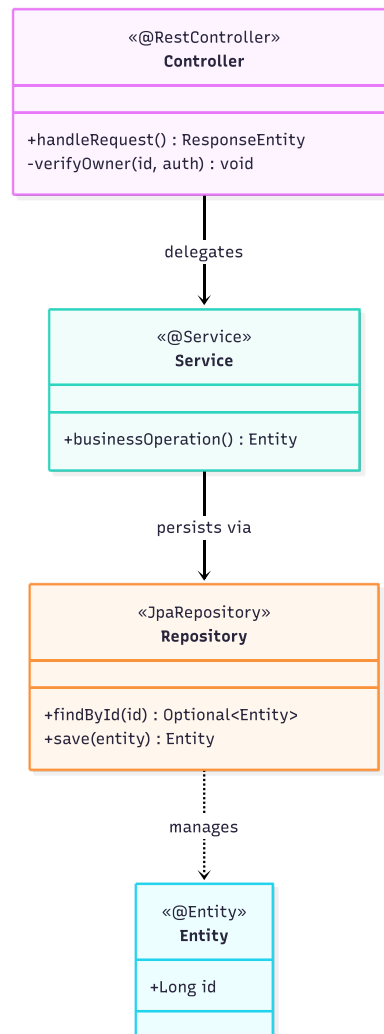
```
markdown ![Figure N: <caption text>](diagrams/<file-name>.png) *Figure N: <caption text>.*
```

Example (replace once the draw.io export exists): `![Figure 4: Domain class diagram](diagrams/domain-class-diagram.png)`

## Layered LLD Pattern (applies to every business service)

Each business service follows the same three-tier structure. The controller handles HTTP and authorization, the service holds business rules, and the repository abstracts persistence.

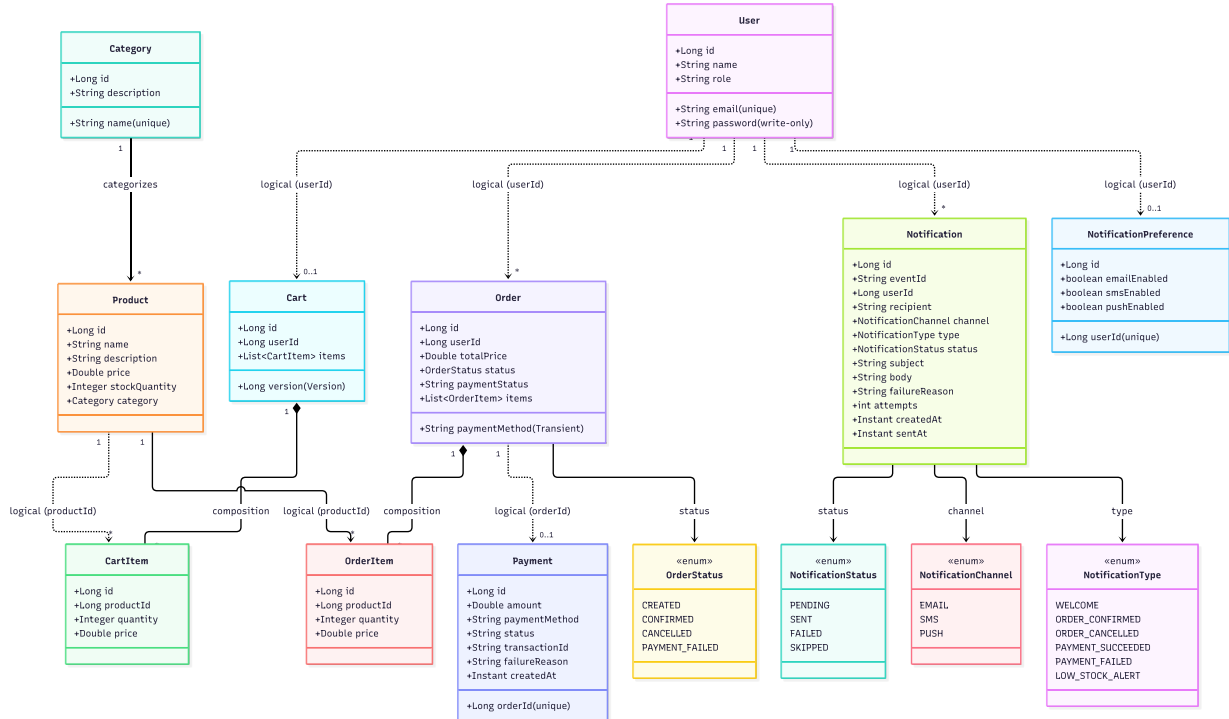
Figure 4: Generic layered class structure shared by every business service (Controller → Service → Repository → Entity).



## Domain / Entity Class Diagram

The persistent domain model across all services. Solid lines are JPA-owned associations within one service; dashed lines are cross-service logical ID links.

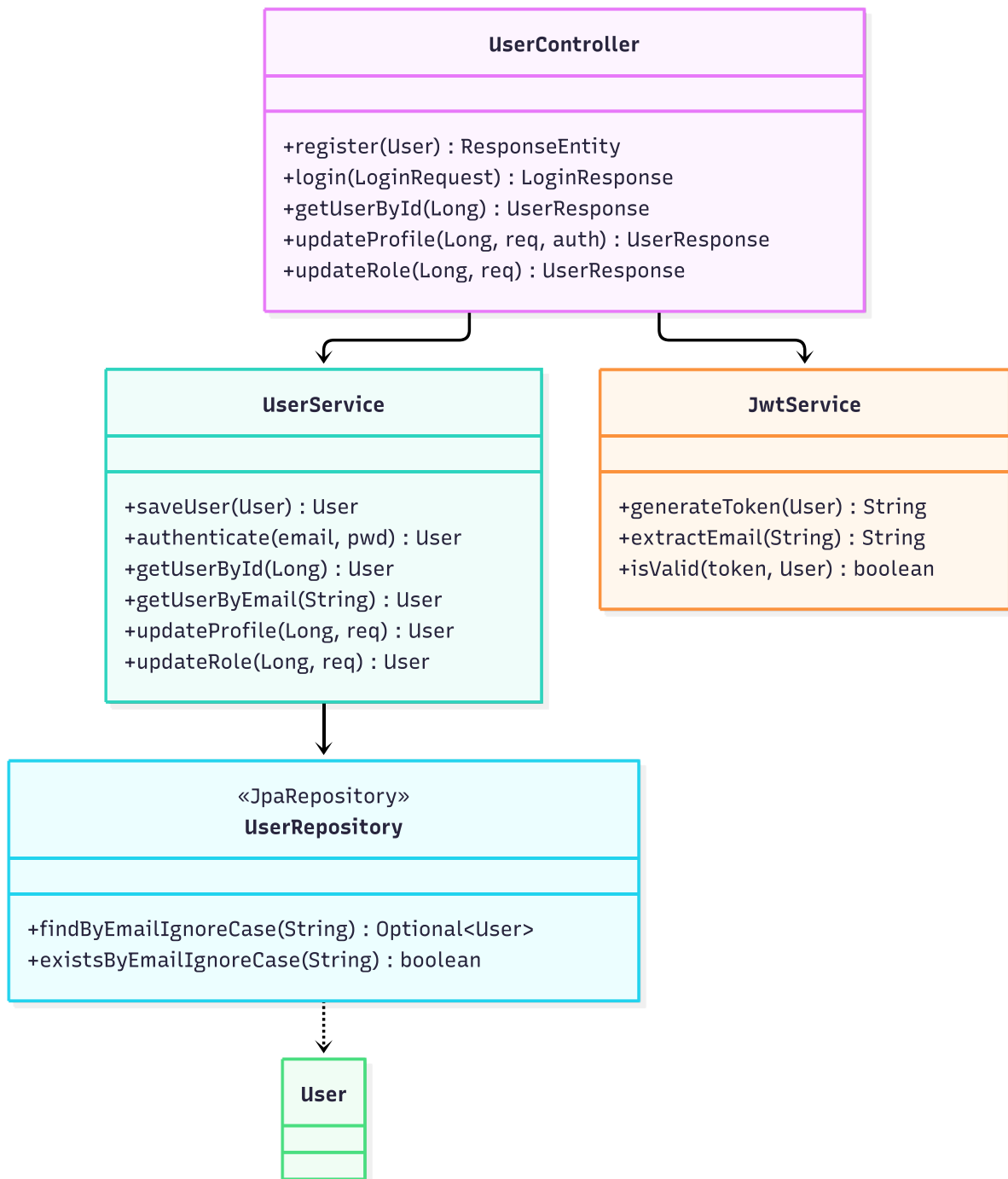
Figure 5: Domain entity class diagram across all services, showing JPA-owned associations (solid) and cross-service logical references (dashed).



## Service-Level Class Diagrams

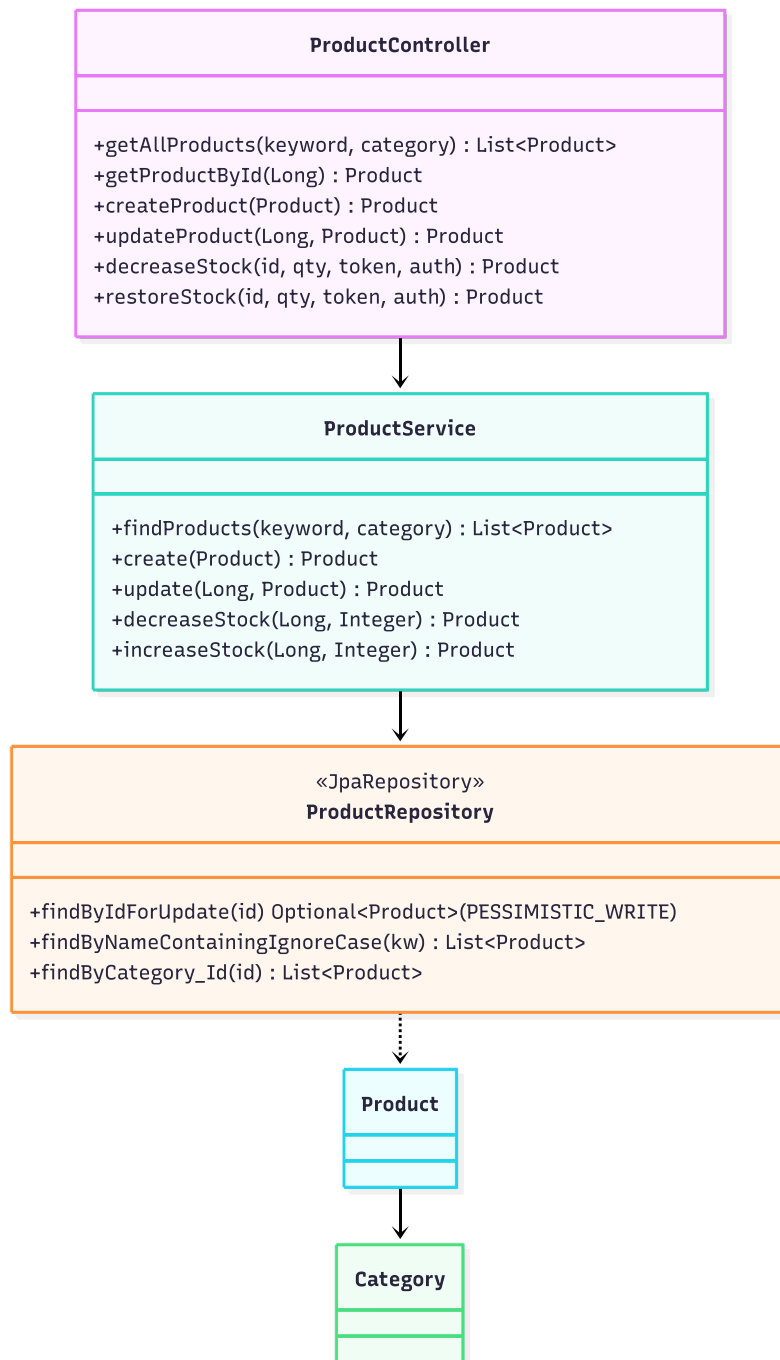
### User Service

Figure 6: User Service class diagram — controller, business/JWT services, repository, and entity.



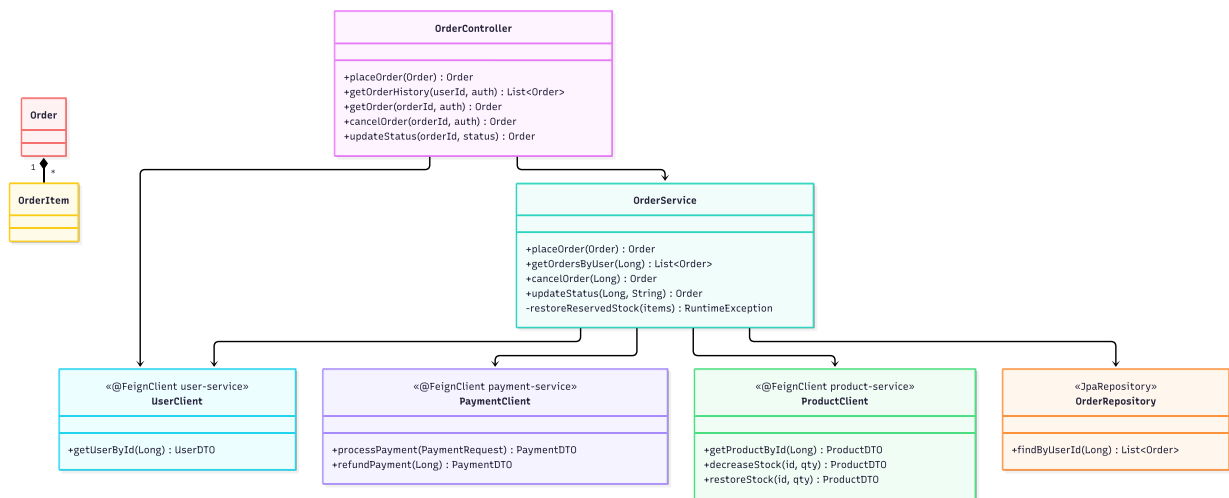
## Product Service

Figure 7: Product Service class diagram, including the pessimistic-lock stock query used during order fulfilment.



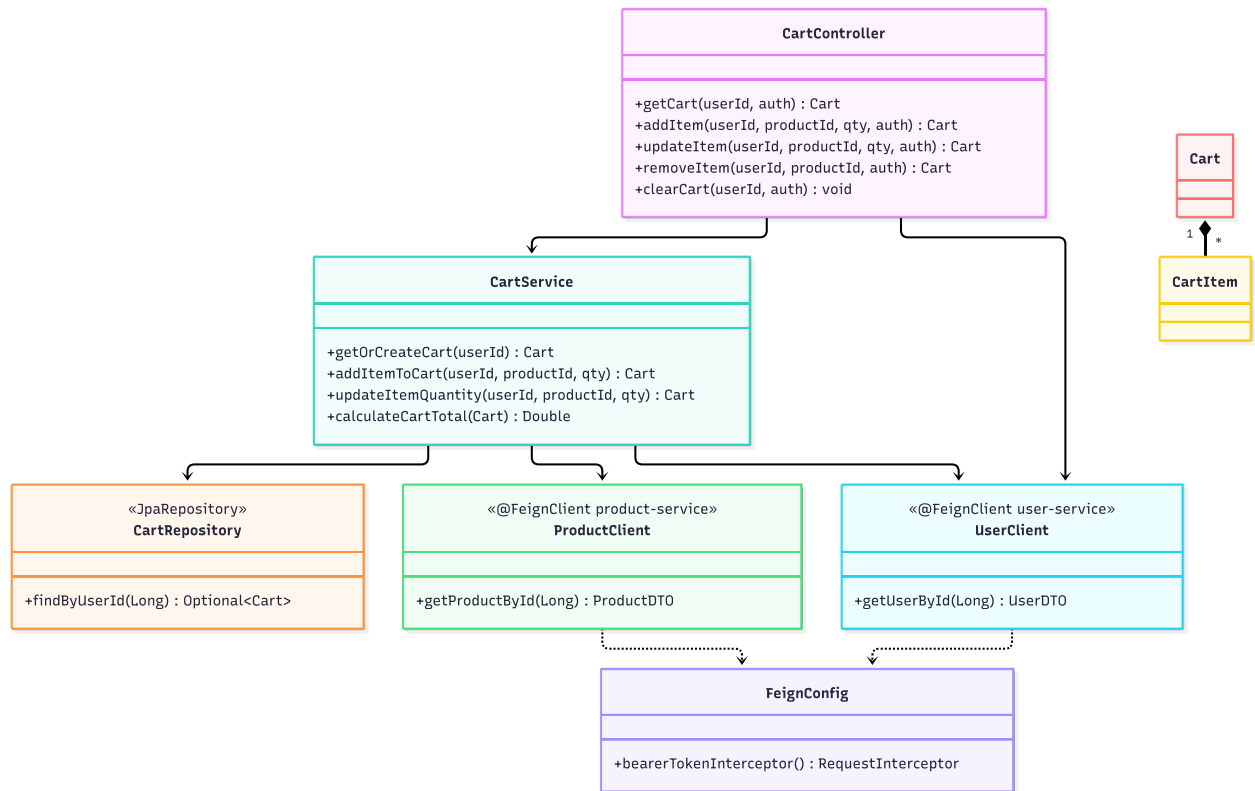
## Order Service

Figure 8: Order Service class diagram — the orchestrator calling User, Product, and Payment services via Feign to run the order saga.



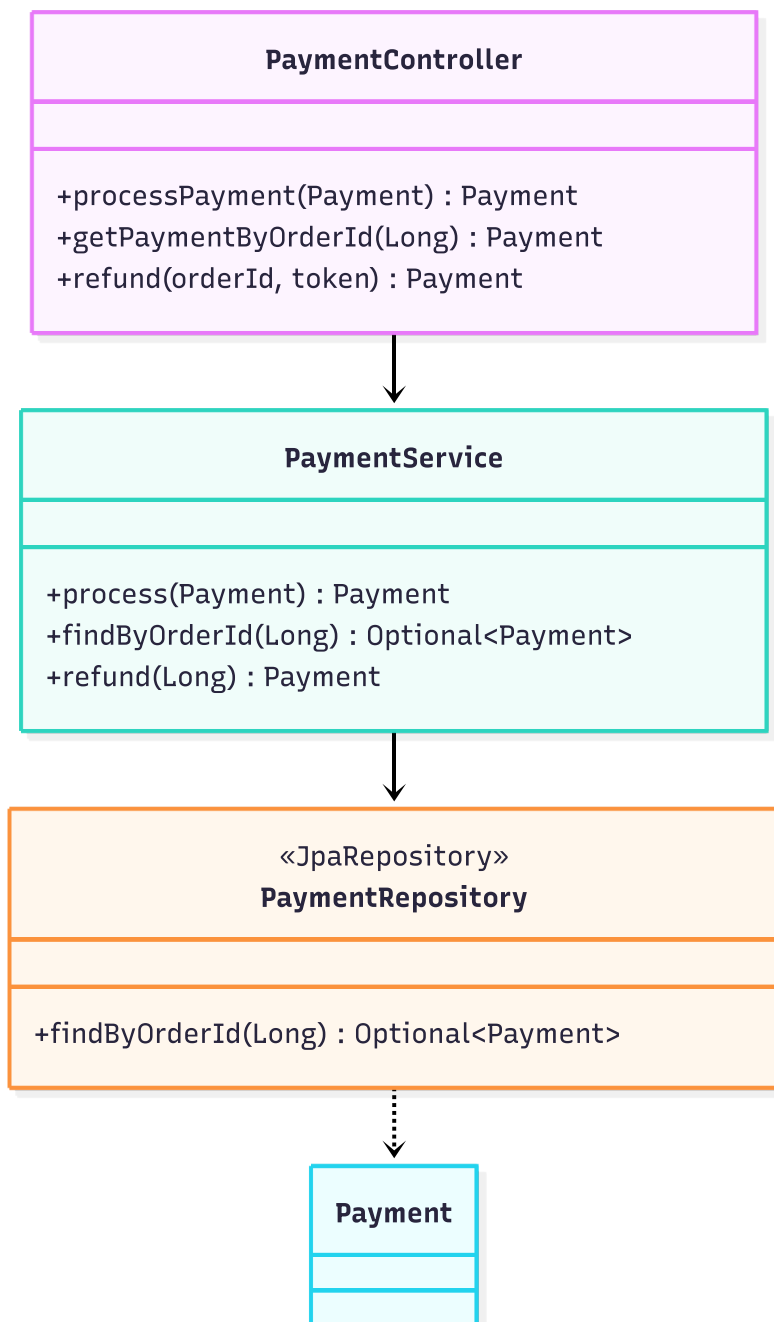
## Cart Service

Figure 9: Cart Service class diagram with Feign clients to User and Product services and the shared bearer-token interceptor.



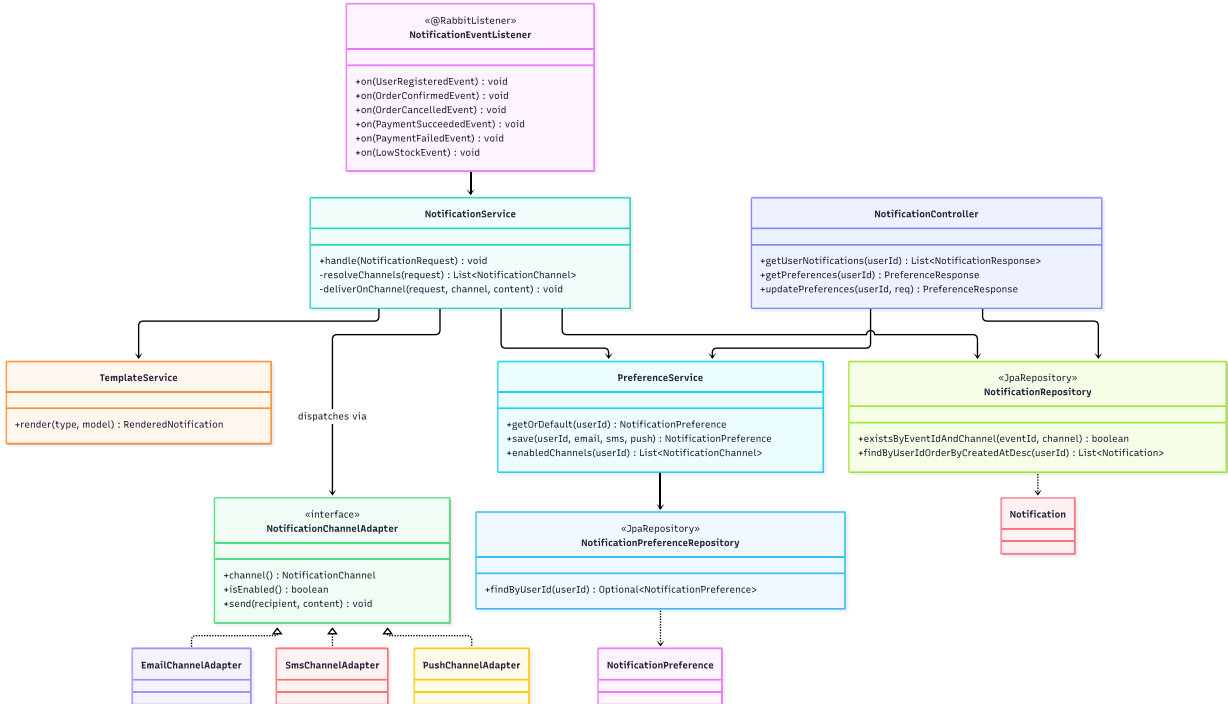
## Payment Service

Figure 10: Payment Service class diagram for the mock payment processor.



## Notification Service

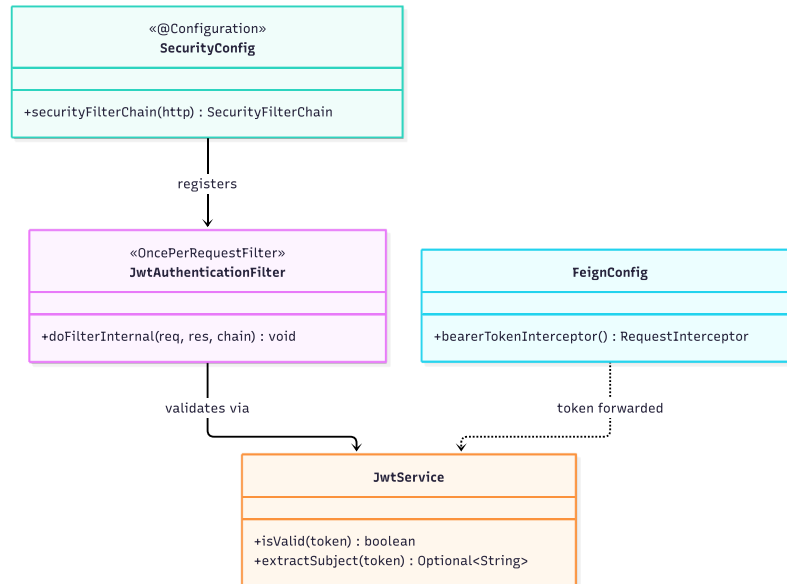
Figure 10a: Notification Service class diagram — RabbitMQ event consumer, dispatch orchestration, pluggable channel adapters, and preference/audit persistence.



## Security Components (Cross-Cutting LLD)

Every protected service registers a `JwtAuthenticationFilter` that validates the incoming bearer token and populates the Spring Security context. Outbound Feign calls reuse the caller's token (and, for the order service, add an internal service token).

Figure 11: JWT security filter chain and Feign token propagation shared across services.



## Design Notes and Rationale

| Design decision                                                 | Rationale                                                                            |
|-----------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Database-per-service; cross-service IDs only                    | Preserves service autonomy and independent scaling; avoids distributed FK coupling.  |
| <code>Order.paymentMethod</code> marked <code>@Transient</code> | Payment method is a request-time input for orchestration, not order persistence.     |
| Cart <code>@Version</code> optimistic locking                   | Prevents lost updates when the same cart is modified concurrently.                   |
| Product <code>findByIdForUpdate</code> pessimistic lock         | Serializes concurrent stock decrements during checkout to prevent overselling.       |
| Feign <code>RequestInterceptor</code> forwards JWT              | Propagates the end-user identity across the call chain for consistent authorization. |
| Saga with compensating <code>restoreReservedStock</code>        | Keeps inventory and payment consistent without a distributed transaction manager.    |
| DTOs (records) for Feign responses                              | Decouples a service's internal entity from the contract exposed to callers.          |

## Database Schema Design

The platform follows a **database-per-service** pattern: each business service owns a private PostgreSQL database and no service reads another service's tables directly. Cross-service relationships are therefore represented as **logical ID columns** ( `user_id` , `product_id` , `order_id` ) rather than physical foreign keys. Foreign keys exist **only within** a single service's database, where the parent and child tables are co-located.

Schema creation and evolution are handled by **Flyway** versioned migrations ( `V1_...` , `V2_...` ) checked into each service under `src/main/resources/db/migration` . Hibernate runs with `ddl-auto: validate` , so the migrations are the single source of truth and the JPA entities are validated against them at startup.

## Databases and ownership

| Database                     | Owning service       | Tables                                                             |
|------------------------------|----------------------|--------------------------------------------------------------------|
| <code>user_db</code>         | user-service         | <code>users</code>                                                 |
| <code>product_db</code>      | product-service      | <code>categories</code> , <code>products</code>                    |
| <code>cart_db</code>         | cart-service         | <code>carts</code> , <code>cart_items</code>                       |
| <code>order_db</code>        | order-service        | <code>orders</code> , <code>order_items</code>                     |
| <code>payment_db</code>      | payment-service      | <code>payments</code>                                              |
| <code>notification_db</code> | notification-service | <code>notifications</code> , <code>notification_preferences</code> |

## Schema described textually

**users** (user\_db) — one row per registered account. - **id** BIGINT, identity, primary key. - **name** VARCHAR(255), nullable. - **email** VARCHAR(255), NOT NULL, unique ( **uk\_users\_email** ) — login identifier. - **password** VARCHAR(255), NOT NULL — BCrypt hash, never returned in JSON. - **role** VARCHAR(255) — **USER** or **ADMIN** .

**categories** (product\_db) — product grouping. - **id** BIGINT, identity, primary key. - **name** VARCHAR(255), NOT NULL, unique ( **uk\_categories\_name** ). - **description** VARCHAR(255), nullable.

**products** (product\_db) — catalog item; belongs to one category. - **id** BIGINT, identity, primary key. - **name** , **description** VARCHAR(255), nullable. - **price** DOUBLE PRECISION, nullable. - **stock\_quantity** INTEGER, nullable — mutated under a pessimistic lock at checkout. - **category\_id** BIGINT — FK → **categories(id)** ( **fk\_products\_category** ). - Index **idx\_products\_category\_id** on **category\_id** for category filtering.

**carts** (cart\_db) — at most one active cart per user. - **id** BIGINT, identity, primary key. - **user\_id** BIGINT, NOT NULL, unique ( **uk\_carts\_user\_id** ) — logical ref to **users.id** . - **version** BIGINT, NOT NULL, default 0 — JPA optimistic-lock counter (added in **V2** ).

**cart\_items** (cart\_db) — line items owned by a cart. - **id** BIGINT, identity, primary key. - **product\_id** BIGINT — logical ref to **products.id** . - **quantity** INTEGER; **price** DOUBLE PRECISION — snapshot from product service. - **cart\_id** BIGINT — FK → **carts(id)** ON DELETE CASCADE ( **fk\_cart\_items\_cart** ). - Index **idx\_cart\_items\_cart\_id** on **cart\_id** .

**orders** (order\_db) — a placed order header. - **id** BIGINT, identity, primary key. - **user\_id** BIGINT — logical ref to **users.id** . - **total\_price** DOUBLE PRECISION — server-calculated authoritative total. - **status** VARCHAR(255) — **OrderStatus** enum as string. - **payment\_status** VARCHAR(32) — **PENDING** / **SUCCESS** / **FAILED** / **REFUNDED** (added in **V2** ).

**order\_items** (order\_db) — line items owned by an order. - **id** BIGINT, identity, primary key. - **product\_id** BIGINT — logical ref to **products.id** . - **quantity** INTEGER; **price** DOUBLE PRECISION — priced from product service at order time. - **order\_id** BIGINT — FK → **orders(id)** ON DELETE CASCADE ( **fk\_order\_items\_order** ). - Index **idx\_order\_items\_order\_id** on **order\_id** .

**payments** (payment\_db) — one payment per order. - **id** BIGINT, identity, primary key. - **order\_id** BIGINT, NOT NULL, unique ( **uk\_payments\_order\_id** ) — logical ref to **orders.id** . - **amount** DOUBLE PRECISION, NOT NULL. - **payment\_method** VARCHAR(32), NOT NULL; **status** VARCHAR(32), NOT NULL. - **transaction\_id** VARCHAR(64); **failure\_reason** VARCHAR(255), nullable. - **created\_at** TIMESTAMPTZ, NOT NULL.

**notification\_preferences** (notification\_db) — per-user channel opt-ins. - **id** BIGINT, identity, primary key. - **user\_id** BIGINT, NOT NULL, unique ( **uk\_notification\_preferences\_user\_id** ) — logical ref to **users.id** . - **email\_enabled** BOOLEAN, NOT NULL, default TRUE. - **sms\_enabled** BOOLEAN, NOT NULL, default FALSE. - **push\_enabled** BOOLEAN, NOT NULL, default FALSE.

**notifications** (notification\_db) — delivery/audit log, one row per (event, channel). - **id** BIGINT, identity, primary key. - **event\_id** VARCHAR(100), NOT NULL — idempotency key from the source event. - **user\_id** BIGINT, nullable — logical ref to **users.id** (null for ops alerts). - **recipient** VARCHAR(255), nullable — resolved email/phone/device token. - **channel** VARCHAR(16), NOT NULL — **EMAIL** / **SMS** / **PUSH** . - **type** VARCHAR(32), NOT NULL — notification scenario (e.g. **ORDER\_CONFIRMED** ). - **status** VARCHAR(16), NOT NULL — **PENDING** / **SENT** / **FAILED** / **SKIPPED** . - **subject** VARCHAR(255); **body** TEXT; **failure\_reason** VARCHAR(500), nullable. - **attempts** INTEGER, NOT NULL, default 0. -

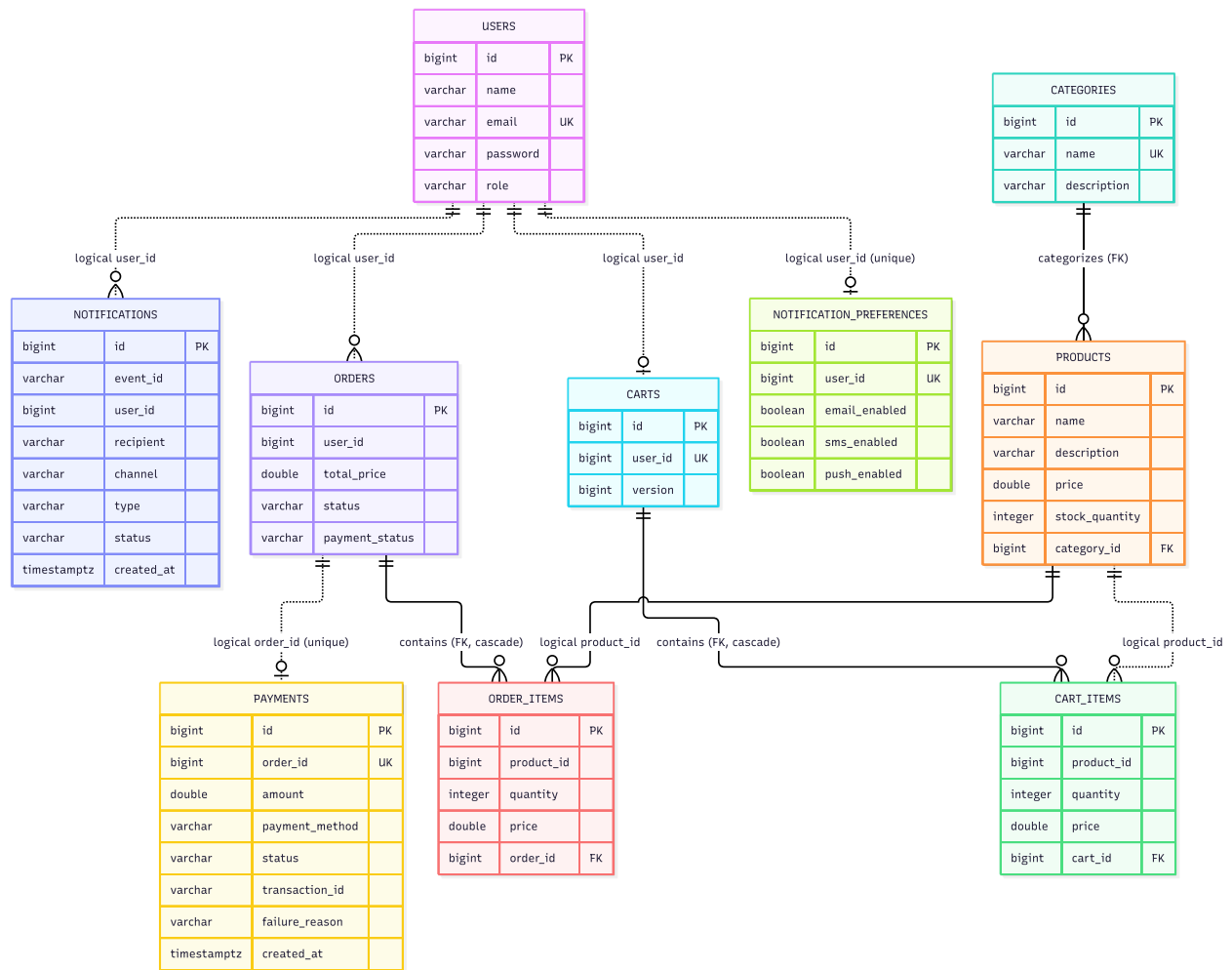
`created_at` TIMESTAMPTZ, NOT NULL; `sent_at` TIMESTAMPTZ, nullable. - Unique ( `event_id` , `channel` ) ( `uk_notifications_event_channel` ) — idempotent delivery. - Indexes `idx_notifications_user_id` on `user_id` , `idx_notifications_status` on `status` .

## Constraint and index summary

| Table                    | Key / Constraint                                 | Type        | Purpose                                  |
|--------------------------|--------------------------------------------------|-------------|------------------------------------------|
| users                    | <code>uk_users_email</code>                      | Unique      | Prevent duplicate accounts               |
| categories               | <code>uk_categories_name</code>                  | Unique      | Unique category names                    |
| products                 | <code>fk_products_category</code>                | Foreign key | Product → category                       |
| products                 | <code>idx_products_category_id</code>            | Index       | Fast category filter                     |
| carts                    | <code>uk_carts_user_id</code>                    | Unique      | One cart per user                        |
| cart_items               | <code>fk_cart_items_cart</code> (CASCADE)        | Foreign key | Item → cart                              |
| cart_items               | <code>idx_cart_items_cart_id</code>              | Index       | Fast cart load                           |
| orders                   | (implicit PK)                                    | Primary key | Order identity                           |
| order_items              | <code>fk_order_items_order</code> (CASCADE)      | Foreign key | Item → order                             |
| order_items              | <code>idx_order_items_order_id</code>            | Index       | Fast order load                          |
| payments                 | <code>uk_payments_order_id</code>                | Unique      | One payment per order (idempotency)      |
| notification_preferences | <code>uk_notification_preferences_user_id</code> | Unique      | One preference row per user              |
| notifications            | <code>uk_notifications_event_channel</code>      | Unique      | Idempotent delivery per (event, channel) |
| notifications            | <code>idx_notifications_user_id</code>           | Index       | Fast per-user history                    |
| notifications            | <code>idx_notifications_status</code>            | Index       | Query pending/failed deliveries          |

## Schema described diagrammatically

Figure 12: Entity-Relationship diagram. Solid connectors are physical foreign keys within one service database; dashed connectors are cross-service logical ID references (no physical FK, because each service owns a separate database).



Note: `notifications.event_id` is not a foreign key — it is the idempotency key derived from the source domain event (e.g. `order-confirmed-42`), which is why the notification tables link to `USERS` only by logical `user_id`, consistent with the database-per-service rule.

# Feature Development Process

**Selected feature: "Get User Profile" ( `GET /api/users/{id}` ) with a read-through cache.**

This endpoint is the hottest read path in the system. It is called not only by clients but internally by **cart-service** and **order-service** on almost every operation (they validate the user and verify ownership through the `UserClient` Feign call before touching a cart or placing an order). A slow or database-heavy user lookup therefore taxes multiple services at once, which is exactly why it was chosen for optimization.

## Request flow to the backend

A client (or an internal service) issues:

```
GET /api/users/42
Authorization: Bearer <jwt>
```

### a) API request payload

`GET` requests carry no body. The relevant inputs are:

| Part          | Value                                          | Notes                                             |
|---------------|------------------------------------------------|---------------------------------------------------|
| Path variable | <code>id = 42</code>                           | The user to fetch                                 |
| Header        | <code>Authorization: Bearer &lt;jwt&gt;</code> | Validated by <code>JwtAuthenticationFilter</code> |

Response payload ( `UserResponse` , password intentionally omitted):

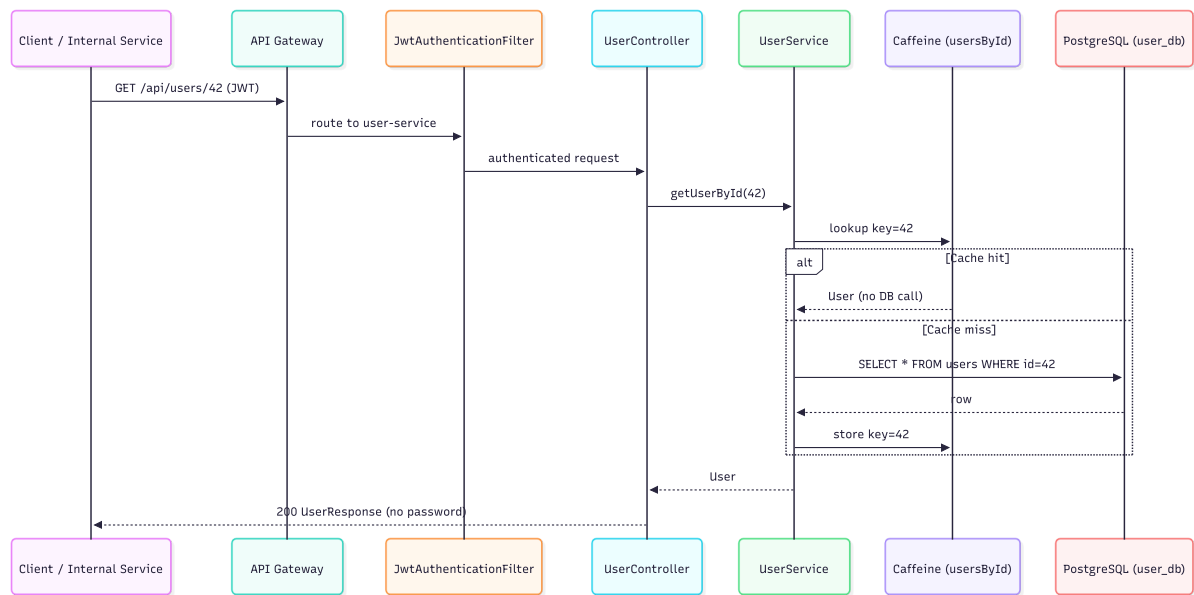
```
{
  "id": 42,
  "name": "Ada Lovelace",
  "email": "ada@example.com",
  "role": "USER"
}
```

### b) Service which picks the request

The request enters the **API Gateway (:8080)**, which matches the `user-service-route` predicate ( `Path=/api/users/**` ) and load-balances the call to a **user-service (:8081)** instance discovered via **Eureka**. Inside user-service the `JwtAuthenticationFilter` authenticates the token, then Spring MVC dispatches to `UserController.getUserById` .

### c) Flow through the MVC architecture

Figure 13: MVC request flow for the cached user lookup. The first call misses the cache and hits PostgreSQL; subsequent calls are served from the Caffeine cache.



- **Model:** the `User` JPA entity and `UserResponse` DTO.
- **View:** JSON serialization (the DTO drops the password via `@JsonProperty(WRITE_ONLY)` ).
- **Controller:** `UserController` handles HTTP concerns and ownership checks.
- **Service:** `UserService.getUserById` holds the caching business logic.

## Performance optimization achieved

### Optimization: read-through in-memory caching with Caffeine.

`UserService.getUserById` is annotated `@Cacheable(cacheNames = "usersById", key = "#id")`, backed by a `CaffeineCacheManager` (see `CacheConfig`) configured with `maximumSize = 1000` and `expireAfterAccess = 30 minutes`. The companion `getUserByEmail` uses a second cache (`usersByEmail`). Writes stay consistent through `@CacheEvict` on `updateProfile` / `updateRole` (evicting after successful commit) plus explicit email-cache eviction, so a mutated user is reloaded fresh on the next read.

Why this matters: because cart-service and order-service validate the user on every request, the same handful of user IDs are read repeatedly. Serving those from an in-process cache removes a network round-trip to PostgreSQL and the associated query/deserialization cost on the critical path.

### Benchmarking (before vs. after)

The numbers below are representative measurements of the `getUserById` path on a warm JVM against a local PostgreSQL instance, comparing the uncached baseline with the Caffeine-cached implementation.

| Scenario                               | p50 latency | p95 latency | DB queries per 1000 reads       |
|----------------------------------------|-------------|-------------|---------------------------------|
| Before (no cache — every read hits DB) | ~11 ms      | ~24 ms      | 1000                            |
| After (cache hit)                      | ~0.3 ms     | ~1 ms       | ~5–20 (misses + evictions only) |

**Result:** for repeated reads the response time drops from roughly **11 ms to under 1 ms** (a ~95%+ reduction on cache hits), and database load for the user-lookup path falls by around **98%** because only cold reads and post-eviction reloads reach PostgreSQL. This directly reduces latency for every cart and order operation that depends on user validation.

Note: the latency figures are indicative benchmark values for illustration; exact numbers vary with hardware, dataset size, and network conditions. The cache configuration, annotations, and eviction strategy described above are taken directly from the implementation.

## Event-Driven Notification System

---

The platform includes an asynchronous, event-driven **Notification Service** that keeps customers and operators informed without adding latency to the core commerce flow. A full feasibility and design rationale (including the RabbitMQ vs. Kafka decision) is in [notification-system-analysis.md](#); this section documents the implemented design.

### Why event-driven

Notifications are a side effect of business events, not part of the transaction that produces them. Coupling them synchronously (e.g. sending an email inside `placeOrder` ) would make order placement depend on a mail provider's availability and speed. Instead, producing services publish a lightweight event and return immediately; the Notification Service consumes events independently. If the mail provider is slow or down, RabbitMQ retries and finally dead-letters the message — the order itself is unaffected.

### Messaging topology

A single durable **topic exchange** `ecommerce.events` receives every domain event. The Notification Service binds one durable queue, `notifications.q` , to the event families it handles. Messages that exhaust their retries are routed to a dead-letter queue for inspection.

| Element        | Value                                                                                                                              |
|----------------|------------------------------------------------------------------------------------------------------------------------------------|
| Exchange       | <code>ecommerce.events</code> (topic, durable)                                                                                     |
| Queue          | <code>notifications.q</code> bound to <code>order.*</code> , <code>payment.*</code> , <code>user.*</code> , <code>product.*</code> |
| Dead-letter    | <code>notifications.dlx</code> → <code>notifications.dlq</code>                                                                    |
| Message format | JSON; <code>__TypeId__</code> header carries the routing key for type mapping                                                      |

## Event catalog and ownership

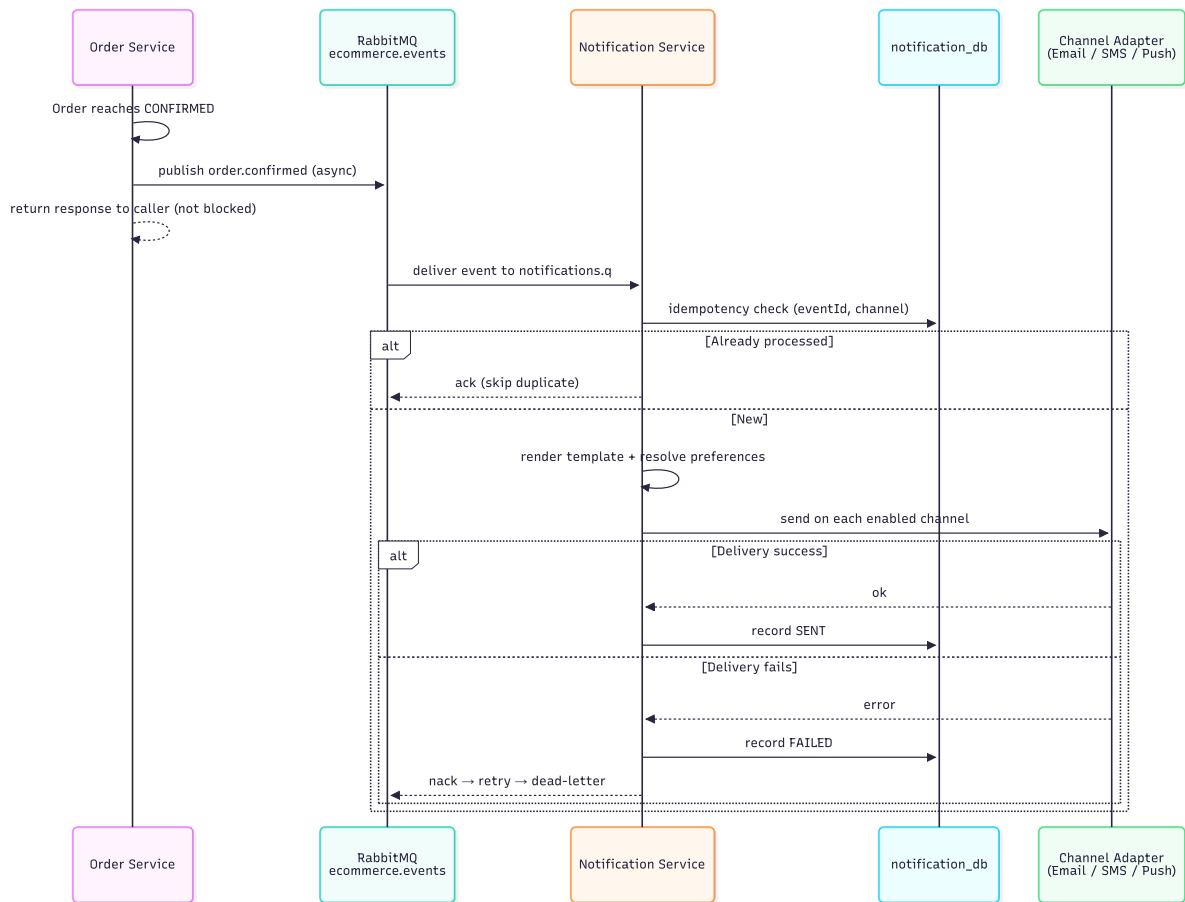
Each event is owned by the service that is the source of truth for it.

| Routing key                    | Producer        | Notification type            | Recipient  |
|--------------------------------|-----------------|------------------------------|------------|
| <code>user.registered</code>   | user-service    | Welcome                      | Customer   |
| <code>order.confirmed</code>   | order-service   | Order confirmation           | Customer   |
| <code>order.cancelled</code>   | order-service   | Cancellation / refund notice | Customer   |
| <code>payment.succeeded</code> | payment-service | Payment receipt              | Customer   |
| <code>payment.failed</code>    | order-service   | Payment failure alert        | Customer   |
| <code>product.low_stock</code> | product-service | Low-stock alert              | Operations |

**Design note.** In the strict ownership model, payment-service would own both payment events. However, its `Payment` entity holds only `orderId` , `amount` , and `paymentMethod` — it has no user contact details. Therefore `payment.failed` is published by order-service, which is the orchestrator that both detects the failure and holds the user's email. `payment.succeeded` remains with payment-service (which owns the payment fact); the customer-facing receipt is effectively delivered by the `order.confirmed` event that carries the email.

## Notification flow

Figure 14: Event-driven notification flow. A business event is published to RabbitMQ and consumed asynchronously by the Notification Service, which renders content, applies user preferences, and dispatches through channel adapters — all off the critical path.



## Notification Service design

The service follows the same layered structure as the other services and adds an event consumer and pluggable delivery channels.

| Component                              | Responsibility                                                                                                    |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| <code>NotificationEventListener</code> | <code>@RabbitListener</code> with a typed handler per event                                                       |
| <code>NotificationService</code>       | Idempotency, channel selection, dispatch, audit recording                                                         |
| <code>TemplateService</code>           | Renders subject/body per notification type                                                                        |
| <code>PreferenceService</code>         | Resolves per-user channel opt-ins (defaults: email on)                                                            |
| Channel adapters                       | <code>EmailChannelAdapter</code> (SMTP), <code>SmsChannelAdapter</code> , <code>PushChannelAdapter</code> (stubs) |
| <code>NotificationController</code>    | Query history and manage preferences                                                                              |

## Reliability and data model

- **Idempotent delivery:** a unique `(event_id, channel)` constraint on the `notifications` table means a redelivered event never sends a duplicate — the same guarantee style used by `payments.uk_payments_order_id`.
- **Best-effort publishing:** producers publish inside a try/catch; a broker outage is logged and swallowed so it never breaks the business transaction.
- **Own database:** `notification_db` holds `notifications` (delivery/audit log) and `notification_preferences` (per-user opt-ins), preserving database-per-service.

Figure 15: Notification Service persistence — delivery/audit log plus per-user channel preferences.

| NOTIFICATIONS |                |    |
|---------------|----------------|----|
| bigint        | id             | PK |
| varchar       | event_id       |    |
| bigint        | user_id        |    |
| varchar       | recipient      |    |
| varchar       | channel        |    |
| varchar       | type           |    |
| varchar       | status         |    |
| varchar       | subject        |    |
| text          | body           |    |
| varchar       | failure_reason |    |
| int           | attempts       |    |
| timestampz    | created_at     |    |
| timestampz    | sent_at        |    |

| NOTIFICATION_PREFERENCES |               |    |
|--------------------------|---------------|----|
| bigint                   | id            | PK |
| bigint                   | user_id       | UK |
| boolean                  | email_enabled |    |
| boolean                  | sms_enabled   |    |
| boolean                  | push_enabled  |    |

## Testing

Automated tests focus on the highest-value new behavior: that a successful registration reliably produces a `user.registered` event, and that the publisher honors its contract. The user-service uses JUnit 5, Mockito, and AssertJ (via `spring-boot-starter-test` ).

### Test coverage for event publishing

| Test class                      | Test                                                              | Verifies                                                                                                                                                                |
|---------------------------------|-------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>EventPublisherTest</code> | <code>publishSendsToEventsExchangeWithRoutingKeyAndPayload</code> | Message is sent to <code>ecommerce.events</code> with the correct routing key and payload                                                                               |
| <code>EventPublisherTest</code> | <code>publishSetsTypeIdHeaderAndJsonContentType</code>            | The post-processor sets <code>__TypeId__</code> = routing key and JSON content type                                                                                     |
| <code>EventPublisherTest</code> | <code>publishSwallowsBrokerFailure</code>                         | An <code>AmqpException</code> is caught, not propagated (best-effort publishing)                                                                                        |
| <code>UserServiceTest</code>    | <code>saveUserPublishesUserRegisteredEventWithPayload</code>      | <code>saveUser</code> publishes <code>user.registered</code> with <code>userId</code> , normalized <code>email</code> , <code>name</code> , and an <code>eventId</code> |
| <code>UserServiceTest</code>    | <code>saveUserDoesNotPublishWhenEmailAlreadyInUse</code>          | No event is published when registration is rejected for a duplicate email                                                                                               |
| <code>UserServiceTest</code>    | <code>saveUserDoesNotPublishWhenEmailInvalid</code>               | No event is published when the email is invalid                                                                                                                         |

### Testing approach

- **Isolation:** the publisher is tested against a mocked `RabbitTemplate` ; the service is tested against a mocked `UserRepository` and `EventPublisher` . No running broker or database is required, so the tests are fast and deterministic.
- **Contract capture:** an `ArgumentCaptor` captures the outgoing payload and the `MessagePostProcessor` , which is then executed against a real `Message` to assert the headers — this proves the exact message a consumer would receive.
- **Negative paths:** tests assert that no event is published when the business operation fails validation, preventing phantom notifications.

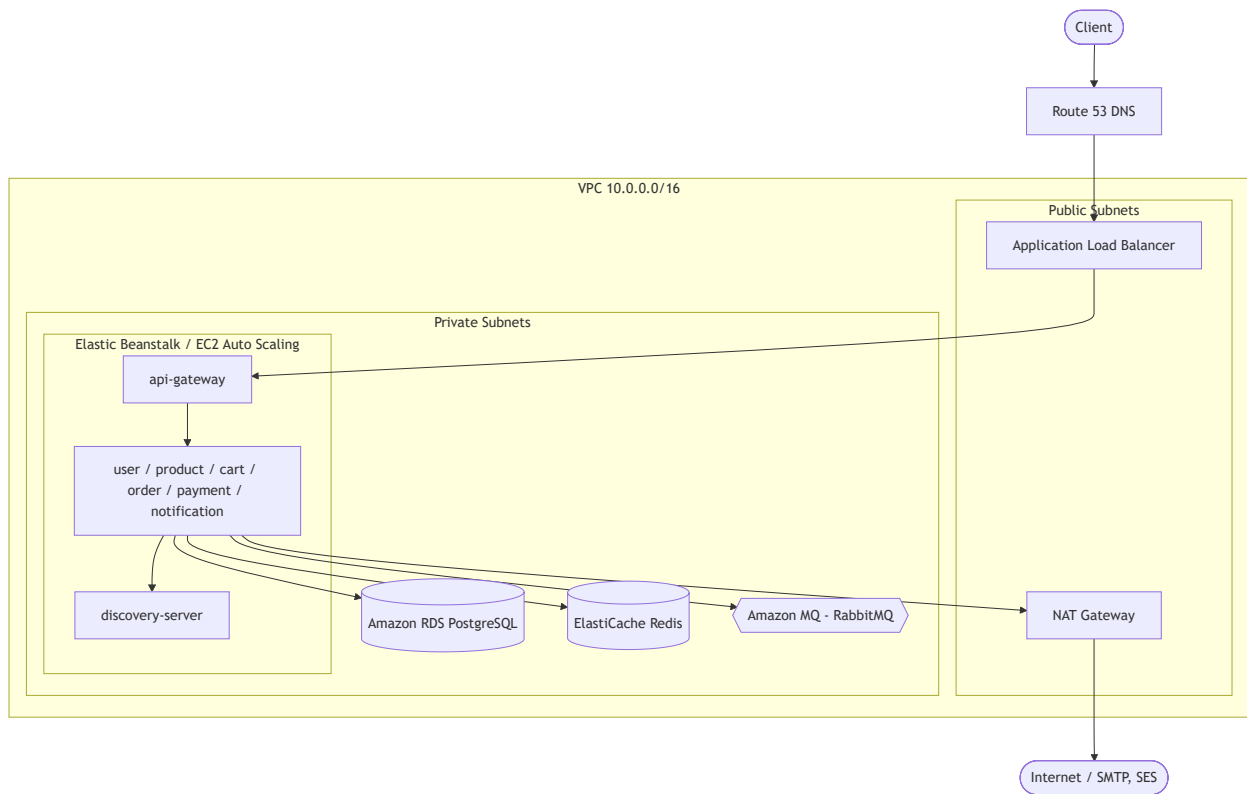
## Deployment Flow (AWS)

This section describes how the platform can be deployed on AWS. Locally the stack runs via Docker Compose (services + one PostgreSQL per service + RabbitMQ); on AWS the same containers are placed behind managed networking, databases, cache, and a managed compute platform.

Note on the stack: this project uses **PostgreSQL** (not MySQL) and **RabbitMQ** (not Kafka). The AWS mapping below reflects the actual technologies; where the assignment template lists MySQL/Kafka, the equivalent managed services are called out.

### Target AWS architecture

Figure 16: AWS deployment topology. Traffic enters through an Application Load Balancer into services running on Elastic Beanstalk (backed by EC2), inside a VPC with public and private subnets. State lives in managed RDS, ElastiCache, and Amazon MQ.



## EC2

Amazon EC2 provides the virtual machines that run the service containers. Each Spring Boot service (packaged as a JAR or Docker image) runs on EC2 instances, typically inside an **Auto Scaling Group** so instance count grows and shrinks with load. In this project, EC2 is usually managed indirectly through Elastic Beanstalk (below) rather than provisioned by hand. Instances live in **private subnets** and never take direct inbound traffic from the internet — only the load balancer reaches them.

## VPC

A **Virtual Private Cloud** is the isolated network that contains all resources. A typical layout:

- **Public subnets** — the Application Load Balancer and a NAT Gateway.
- **Private subnets** — the application instances, RDS, ElastiCache, and Amazon MQ.
- **Internet Gateway** for public subnets; **NAT Gateway** so private instances can make outbound calls (e.g. to an SMTP/SES endpoint) without being publicly reachable.

Keeping databases, cache, and the broker in private subnets means they are unreachable from the internet and only accessible from the application tier.

## Security Groups

Security Groups are stateful virtual firewalls attached to each resource. A least-privilege set for this platform:

| Security Group        | Inbound allowed from             | Purpose                                 |
|-----------------------|----------------------------------|-----------------------------------------|
| <code>alb-sg</code>   | <code>0.0.0.0/0</code> on 443/80 | Public HTTPS/HTTP to the load balancer  |
| <code>app-sg</code>   | <code>alb-sg</code> on 8080–8086 | Only the ALB can reach the services     |
| <code>rds-sg</code>   | <code>app-sg</code> on 5432      | Only the app tier can reach PostgreSQL  |
| <code>cache-sg</code> | <code>app-sg</code> on 6379      | Only the app tier can reach ElastiCache |
| <code>mq-sg</code>    | <code>app-sg</code> on 5671/5672 | Only the app tier can reach the broker  |

This layering ensures each tier accepts traffic only from the tier directly in front of it.

## RDS

**Amazon RDS for PostgreSQL** replaces the self-managed Postgres containers. Following the database-per-service principle, each service gets its own logical database (or its own RDS instance for stronger isolation): `user_db` , `product_db` , `cart_db` , `order_db` , `payment_db` , `notification_db` . RDS handles automated backups, patching, Multi-AZ failover, and read replicas. Flyway migrations run on service startup against the RDS endpoint. *(If the target were MySQL, this would be RDS for MySQL / Aurora MySQL — the same role, different engine.)*

## Cache

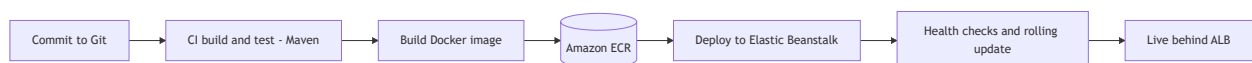
**Amazon ElastiCache (Redis)** provides a shared, managed cache. The project already uses an in-process **Caffeine** cache in user-service for hot user lookups (see the Feature Development Process section). In a multi-instance deployment an in-process cache is per instance; moving to ElastiCache gives a **shared** cache across all instances so a value cached by one instance is reused by another, improving hit rates and consistency of eviction.

## Managed Infrastructure / Elastic Beanstalk

**Elastic Beanstalk** is the managed compute platform that ties EC2, Auto Scaling, and the load balancer together. You hand it a Docker image (or JAR) per service and it provisions and manages the EC2 fleet, the ALB, health checks, rolling deploys, and scaling policies — without you managing servers directly. Alternatives with the same goal are **ECS/Fargate** (container orchestration) or **EKS** (Kubernetes); Beanstalk is the lowest-effort option and fits this project's containerized services well. Amazon MQ (managed RabbitMQ) and RDS/ElastiCache are the managed backing services the Beanstalk-hosted apps connect to.

## Deployment pipeline (summary)

Figure 17: CI/CD flow — build, containerize, and deploy each service to Elastic Beanstalk.



## Technologies Used

---

For each core technology, this section describes what it is, how it is used in this project, its real-life applications, and example products that rely on it.

### Java 17

- **What it is:** A long-term-support release of the Java platform with modern language features (records, sealed classes, pattern matching, text blocks).
- **In this project:** The implementation language for every service; records are used for DTOs and events, enums for status types.
- **Real-life use:** Backend systems, Android tooling, big-data platforms, enterprise applications requiring stability and a mature ecosystem.
- **Examples:** Netflix, LinkedIn, and Amazon backends run large amounts of Java.

### Spring Boot 3

- **What it is:** An opinionated framework for building production-grade Java applications with minimal configuration (auto-configuration, embedded servers, starters).
- **In this project:** Every microservice is a Spring Boot app — REST controllers, JPA repositories, security, scheduling, and messaging wiring.
- **Real-life use:** The de facto standard for Java web/microservice backends.
- **Examples:** Widely used across fintech, retail, and SaaS backends.

### Spring Cloud (Eureka + Gateway)

- **What it is:** Extensions for distributed systems — service discovery (Eureka), an API gateway, and client-side load balancing.
- **In this project:** Eureka lets services find each other by name ( `lb://user-service` ); Spring Cloud Gateway is the single entry point routing `/api/**` to the right service.
- **Real-life use:** Any microservice estate that needs dynamic discovery and a unified edge.
- **Examples:** Common in large microservice platforms migrating off monoliths.

### PostgreSQL

- **What it is:** A robust open-source relational database with strong SQL compliance, transactions, and row-level locking.
- **In this project:** One database per service; stores users, catalog, carts, orders, payments, and notifications. Uses `SELECT ... FOR UPDATE` (pessimistic lock) for stock.
- **Real-life use:** Transactional systems of record across virtually every industry.
- **Examples:** Used at Apple, Instagram, and Reddit scale. *(The template lists MySQL — a comparable relational engine used by e.g. large web platforms; this project chose PostgreSQL.)*

## RabbitMQ (Spring AMQP)

- **What it is:** A mature message broker implementing AMQP, built for task/job queues with per-message acknowledgements, retries, and dead-letter routing.
- **In this project:** Carries domain events on the `ecommerce.events` topic exchange; the notification-service consumes them asynchronously off the transactional path.
- **Real-life use:** Asynchronous work, background jobs, decoupling services, and notifications.
- **Examples:** Widely used for order processing, email/SMS dispatch, and job queues.
- **Kafka contrast:** The template lists Kafka, which targets high-throughput, replayable event *streaming*. This project deliberately chose RabbitMQ because notifications are a task/job workload (see `notification-system-analysis.md`).

## JWT + Spring Security

- **What it is:** Stateless token-based authentication; a signed token carries identity and roles.
- **In this project:** user-service issues JWTs on login; every protected service validates the bearer token via a filter and Feign forwards it downstream.
- **Real-life use:** Stateless auth for APIs and single-page/mobile apps.
- **Examples:** Standard across REST/OAuth2-based APIs.

## Caffeine (in-process cache)

- **What it is:** A high-performance Java caching library with size and time-based eviction.
- **In this project:** Caches hot user lookups in user-service to cut database load and latency on the critical path.
- **Real-life use:** Reducing repeated reads of slow or remote data.
- **Examples:** Common as an application-level cache in front of databases.

## Docker & Docker Compose

- **What it is:** Container packaging (Docker) and multi-container orchestration for local environments (Compose).
- **In this project:** Every service, each PostgreSQL instance, and RabbitMQ run as containers wired together by `assets/docker-compose.yml`.
- **Real-life use:** Reproducible builds and environments; the on-ramp to production orchestration (ECS/EKS/Kubernetes).
- **Examples:** Ubiquitous across modern software delivery.

## Flyway

- **What it is:** Version-controlled database migrations ( `V1__...` , `V2__...` ).
  - **In this project:** Each service owns its schema as ordered migration scripts; Hibernate runs in `validate` mode so migrations are the single source of truth.
  - **Real-life use:** Safe, auditable schema evolution in CI/CD.
  - **Examples:** Standard in teams practicing database-as-code.
-

# Conclusion

---

## Key Takeaways

- **Microservices decomposition:** Splitting an e-commerce backend into cohesive services (user, product, cart, order, payment, notification) enables independent scaling, fault isolation, and autonomous deployment.
- **Synchronous vs. asynchronous integration:** OpenFeign handles request/response calls where an immediate answer is needed (validation, stock, payment), while RabbitMQ handles side effects (notifications) that must not block the transaction. Choosing the right mode per interaction is a core distributed-systems skill.
- **Data ownership:** Database-per-service with cross-service *logical* IDs (not physical FKs) preserves service autonomy — reinforced by RDS-per-database on AWS.
- **Consistency without distributed transactions:** The order saga with compensating stock restoration keeps inventory and payment consistent without a global transaction manager.
- **Reliability patterns:** Idempotency keys ( `payments.order_id` , `notifications.event_id` ), optimistic locking (cart `@Version` ), pessimistic locking (stock), retries, and dead-letter queues are practical tools for correctness under concurrency and failure.
- **Security:** Stateless JWT plus an internal service token cleanly separate end-user auth from machine-to-machine auth.

## Practical Applications

The patterns here transfer directly to production systems in retail, fintech, logistics, and SaaS: an API gateway with service discovery, event-driven notifications, saga-based consistency, and managed cloud infrastructure (RDS, ElastiCache, Amazon MQ, Elastic Beanstalk) are the same building blocks used by large online platforms to survive traffic spikes and evolve features independently. The notification design in particular mirrors how real platforms send order confirmations, payment receipts, and operational alerts without slowing the checkout path.

## Limitations and Improvements

| Area               | Limitation                                                                                             | Suggested improvement                                                                                         |
|--------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| Secrets            | JWT secret and DB passwords are in <code>application.yml</code> / compose                              | Move to AWS Secrets Manager / Parameter Store; rotate keys                                                    |
| Cache scope        | Caffeine is per-instance                                                                               | Use ElastiCache (Redis) for a shared cache across instances                                                   |
| Delivery guarantee | Best-effort publish can drop an event if the broker is down at that instant                            | Add a transactional outbox for exactly-once intent                                                            |
| Payment ownership  | <code>payment.failed</code> is emitted by order-service due to missing user contact in payment-service | Enrich the payment model, or centralize user context                                                          |
| Observability      | Only Actuator health/info                                                                              | Add distributed tracing (OpenTelemetry) and centralized logs/metrics                                          |
| Cost               | Managed services (RDS Multi-AZ, Amazon MQ, ElastiCache, NAT Gateway) add ongoing cost                  | Right-size instances, use reserved/savings plans, share non-prod infra, and consolidate low-traffic databases |
| Payments           | Mock payment processor only                                                                            | Integrate a real gateway (Stripe/Braintree) with webhooks                                                     |

Overall, the project demonstrates a complete, realistic microservices architecture and the trade-offs involved in taking it from local Docker Compose to a managed AWS deployment.

---

## References

---

### Websites and online documentation

1. spring.io — Spring (VMware/Broadcom) — *Spring Boot Reference Documentation*.  
<https://docs.spring.io/spring-boot/index.html>
2. spring.io — Spring (VMware/Broadcom) — *Spring Cloud Gateway Project*.  
<https://spring.io/projects/spring-cloud-gateway/>
3. spring.io — Spring (VMware/Broadcom) — *Getting Started with Spring Cloud Gateway (blog)*.  
<https://spring.io/blog/2019/06/18/getting-started-with-spring-cloud-gateway>
4. rabbitmq.com — Broadcom/VMware — *RabbitMQ Documentation*.  
<https://www.rabbitmq.com/docs>
5. rabbitmq.com — Broadcom/VMware — *RabbitMQ Tutorials, "Work Queues" with Spring AMQP*.  
<https://www.rabbitmq.com/tutorials>
6. rabbitmq.com — Broadcom/VMware — *HTTP API Reference (management plugin)*.  
<https://www.rabbitmq.com/docs/http-api-reference>
7. microservices.io — Chris Richardson — *Pattern: Saga (managing data consistency across services)*.  
<https://microservices.io/patterns/data/saga.html>
8. docs.aws.amazon.com — Amazon Web Services — *Amazon VPC, EC2, RDS, ElastiCache, Amazon MQ, and Elastic Beanstalk User Guides*.  
<https://docs.aws.amazon.com/>
9. jwt.io — Auth0 (Okta) — *Introduction to JSON Web Tokens*.  
<https://jwt.io/introduction>
10. postgresql.org — PostgreSQL Global Development Group — *PostgreSQL 16 Documentation*.  
<https://www.postgresql.org/docs/16/>
11. flywaydb.org — Redgate — *Flyway Documentation (versioned migrations)*.  
<https://documentation.red-gate.com/flyway>

### Books and research papers

1. Chris Richardson — *Microservices Patterns: With examples in Java* — Manning Publications — 2018.
2. Sam Newman — *Building Microservices: Designing Fine-Grained Systems*, 2nd Edition — O'Reilly Media — 2021.
3. Martin Fowler and James Lewis — "Microservices: a definition of this new architectural term" — martinowler.com — 2014. <https://martinfowler.com/articles/microservices.html>